

Introdução à Otimização Matemática

O cálculo diferencial como aliado

Clemerson Menezes

31 de maio de 2026

1 O que é Otimização Matemática?

A otimização matemática consiste no processo de encontrar as entradas ideais (ou o melhor conjunto de decisões) para resolver um problema específico, maximizando ou minimizando uma função objetivo.

Formalmente, um problema de otimização pode ser definido como:

$$\begin{aligned} \min_{\mathbf{x} \in \mathcal{X}} \quad & f(\mathbf{x}) \\ \text{sujeito a} \quad & g_i(\mathbf{x}) \leq 0, \quad i = 1, \dots, m \\ & h_j(\mathbf{x}) = 0, \quad j = 1, \dots, p \end{aligned}$$

onde $f(\mathbf{x})$ é a função objetivo, $\mathbf{x}$ é o vetor de variáveis de decisão, e g_i e h_j representam as restrições do sistema.

Exemplos clássicos de aplicação incluem:

- **Agricultura:** Maximizar a quantidade e a qualidade das colheitas, considerando variáveis como condições do solo e clima.
- **Logística:** Encontrar rotas de distribuição ideais considerando tráfego e distância [cite: 23, 24], com o objetivo de melhorar os tempos de entrega e reduzir custos operacionais.

2 Busca Exaustiva (Força Bruta)

O método de busca exaustiva, frequentemente chamado de "força bruta", envolve a discretização do domínio e a avaliação da função objetivo em todos os pontos possíveis. Por exemplo, calcular o lucro para uma série de quantidades e selecionar aquela que produz o maior valor.

2.1 Vantagens e Desvantagens

- **Vantagens:** Possui implementação simples, não requer software caro e faz exigências mínimas (ou suposições mínimas) sobre as propriedades da função (como diferenciabilidade).
- **Desvantagens:** Sofre da *maldição da dimensionalidade*. Não escala para casos complexos ou problemas com muitas variáveis (multivariados).

3 Otimização Univariada e a Função Objetivo

Uma função objetivo descreve a relação matemática entre as variáveis de entrada e o resultado que se deseja otimizar.

3.1 Exemplo Prático: Manufatura de Móveis

Considere o problema de maximizar o lucro, P , na produção de móveis em função da quantidade fabricada, q . A relação é dada por um modelo quadrático univariado (uma única variável):

$$P(q) = 40q - 0.5q^2 \quad (1)$$

Queremos encontrar o valor de q que maximiza P . A forma de parábola côncava da curva sugere que existe um valor ótimo absoluto (ponto máximo), e quedas no lucro em altas quantidades podem ser explicadas por restrições do mundo real, como falta de funcionários ou equipamento.

3.2 O Papel do Cálculo: Condição Necessária de Primeira Ordem (FONC)

A primeira derivada descreve o comportamento da inclinação da função. O ponto ótimo ocorre onde a função da derivada é igual a zero, configurando um ponto crítico.

Para a Equação 1:

$$\frac{dP}{dq} = 40 - q = 0 \implies q^* = 40 \quad (2)$$

O ótimo encontrado, onde a inclinação é nula, é $q^* = 40$.

3.3 Condição Suficiente de Segunda Ordem (SOSC)

A segunda derivada representa a derivada da derivada (a taxa de mudança da inclinação) e é usada para identificar a concavidade e classificar pontos críticos como mínimos ou máximos.

Regras de classificação:

- Se a 2ª derivada no ponto < 0 : É um Máximo (função estritamente côncava localmente).
- Se a 2ª derivada no ponto > 0 : É um Mínimo (função estritamente convexa localmente).
- Se a 2ª derivada no ponto $= 0$: O teste é inconclusivo (pode ser um ponto de sela).

Aplicando ao nosso exemplo:

$$\frac{d^2P}{dq^2} = \frac{d}{dq}(40 - q) = -1 \quad (3)$$

Como $-1 < 0$, confirmamos formalmente que $q^* = 40$ é um ponto de Máximo global. Substituindo na função original, o lucro máximo é $P(40) = \$800$.

4 Otimização Multivariada

Problemas do mundo real raramente dependem de apenas uma variável. A otimização multivariada lida com funções de múltiplas variáveis.

4.1 Exemplo: Fábrica de Biscoitos (Função Cobb-Douglas)

Considere uma função de produção, F , dependente do trabalho (L , horas inseridas pelos trabalhadores) e do capital (K , horas de operação das máquinas):

$$F(K, L) = K^{0.34} L^{0.66} \quad (4)$$

Esta é uma função multivariada não-linear clássica.

4.2 Vetor Gradiente e Condição de Otimalidade

Enquanto na otimização univariada usamos a derivada simples, aqui utilizamos o vetor gradiente, que agrupa as derivadas parciais. As derivadas parciais medem como a inclinação muda em relação a cada variável isoladamente.

O gradiente ∇F é dado por:

$$\nabla F(K, L) = \begin{bmatrix} \frac{\partial F}{\partial K} \\ \frac{\partial F}{\partial L} \end{bmatrix} = \begin{bmatrix} 0.34 K^{-0.66} L^{0.66} \\ 0.66 K^{0.34} L^{-0.34} \end{bmatrix} \quad (5)$$

Para encontrar os pontos críticos, precisaríamos resolver o sistema $\nabla F(K, L) = \mathbf{0}$. No entanto, ao analisar a função da Equação 4, observamos que à medida que K e L aumentam, a produção F também aumenta indefinidamente. Portanto, neste caso específico, sem restrições de orçamento, a função não possui máximos ou mínimos globais finitos (a resolução algorítmica retornará um conjunto vazio ‘[]’).

5 Limites da Diferenciação

Embora o cálculo diferencial seja uma ferramenta poderosa, os algoritmos baseados em gradiente (como *Gradient Descent* ou métodos de Newton baseados no Hessiano) possuem limitações severas quando as premissas matemáticas falham.

Não é possível aplicar a otimização via diferenciação direta em funções que apresentam:

1. **Falta de Diferenciabilidade (No derivative):** Pontos de "bico" ou quinas onde o limite da derivada não existe (ex: funções modulares).
2. **Descontinuidades (Discontinuities):** Saltos abruptos na função onde o limite lateral esquerdo é diferente do limite lateral direito.
3. **Funções estritamente crescentes ou decrescentes (Increasing or decreasing functions):** Onde a derivada nunca é zero no interior do domínio, exigindo abordagens focadas nos limites (fronteiras) do domínio de busca.

Nestes cenários, são requeridos algoritmos heurísticos, otimização inteira, relaxamentos convexos ou métodos baseados em subgradientes.

6 Implementação em Python usando SymPy

Para demonstrar a aplicação prática da teoria acima, utilizamos a biblioteca `sympy` do Python, que permite realizar computação simbólica. O código abaixo resolve os dois problemas apresentados formalmente.

6.1 Caso 1: Otimização Univariada (Manufatura de Móveis)

Neste bloco, calculamos a primeira e segunda derivadas para encontrar o ponto crítico e classificá-lo como máximo.

```
1 import sympy as sp
2
3 # 1. Definindo a variavel simbolica
4 q = sp.symbols('q')
5
6 # 2. Definindo a funcao objetivo (Lucro)
7 P = 40*q - 0.5*q**2
8 print(f"Funcao Objetivo P(q) = {P}\n")
9
10 # 3. Condicao Necessaria de Primeira Ordem (Primeira Derivada)
11 dP_dq = sp.diff(P, q)
12 print(f"Primeira derivada (dP/dq): {dP_dq}")
13
14 # Encontrando o ponto critico (igualando a derivada a zero)
15 q_opt = sp.solve(dP_dq, q)
16 print(f"Ponto critico encontrado (q*): {q_opt[0]}\n")
17
18 # 4. Condicao Suficiente de Segunda Ordem (Segunda Derivada)
19 d2P_dq2 = sp.diff(dP_dq, q) # Derivada da derivada
20 print(f"Segunda derivada (d^2P/dq^2): {d2P_dq2}")
21
22 # Classificando o ponto
23 valor_segunda_derivada = d2P_dq2.subs(q, q_opt[0])
24
25 if valor_segunda_derivada < 0:
26     print("Conclusao: Como a segunda derivada e negativa (< 0),
27           confirmamos um Ponto de MAXIMO!")
28     lucro_maximo = P.subs(q, q_opt[0])
29     print(f"Lucro Maximo atingido: ${lucro_maximo}")
30 elif valor_segunda_derivada > 0:
31     print("Conclusao: Como a segunda derivada e positiva (> 0),
32           temos um Ponto de MINIMO.")
33 else:
34     print("Conclusao: O teste da segunda derivada e inconclusivo.")
```

6.2 Caso 2: Otimização Multivariada (Fábrica de Biscoitos)

Neste bloco, calculamos as derivadas parciais (o gradiente) da função Cobb-Douglas para identificar possíveis pontos de mínimo ou máximo.

```

1 import sympy as sp
2
3 # 1. Definindo as variaveis simbolicas
4 K, L = sp.symbols('K L')
5
6 # 2. Definindo a funcao objetivo (Producao)
7 F = K**0.34 * L**0.66
8 print(f"Funcao de Producao F(K,L) = {F}\n")
9
10 # 3. Calculando o Vetor Gradiente (Derivadas Parciais)
11 dF_dK = sp.diff(F, K)
12 dF_dL = sp.diff(F, L)
13
14 print("Vetor Gradiente:")
15 print(f"dF/dK: {dF_dK}")
16 print(f"dF/dL: {dF_dL}\n")
17
18 # 4. Buscando Pontos Criticos
19 # Tentamos resolver o sistema onde ambas as derivadas parciais
    sao zero simultaneamente
20 pontos_criticos = sp.solve([dF_dK, dF_dL], (K, L))
21
22 print(f"Pontos criticos encontrados: {pontos_criticos}")
23
24 if not pontos_criticos:
25     print("Conclusao: A lista esta vazia. Nao existem maximos ou
        minimos finitos para esta funcao.")
26     print("A producao cresce indefinidamente a medida que K e L
        aumentam.")

```